LabDesk Project Report

Laboratory equipment checkout and return prototype

Student Chandhana S Gupta	**Institution** Sahrdaya College of Engineering and Technology
Student identifiers SR No. 225044 | Register No. SHR25CS093	**Prepared** 26 October 2026

Project note

LabDesk is a learning prototype prepared with AI coding assistance. The student should review and understand the implementation, confirm all details, and disclose assistance as required by course policy. Its features and results are described as implemented in the delivered prototype; no internship placement or external testing is claimed.

Executive Summary

LabDesk is a browser-based laboratory equipment checkout prototype. It gives students a searchable view of equipment and a short loan request form, while a coordinator can review requests and record returns. The delivered build uses React and Vite for the client, Express and Node.js for the API, and Mongoose models for optional MongoDB persistence. Without a MongoDB connection string, the API runs in demonstration mode with in-memory sample records.

Introduction

Laboratories share instruments across classes, practical sessions, and student projects. Paper registers and informal messages can make availability and due dates difficult to track. LabDesk explores a simple digital workflow for one teaching laboratory: show an equipment register, record a loan request, review its status, and check an item back in.

The application theme and broad checkout workflow were informed by LabFlow, an MIT-licensed project by Milan K B. LabDesk is a separate implementation with its own name, interface, and code. The upstream reference is listed in the README.

Objectives

The prototype aims to make equipment availability visible, collect the purpose and return date for a loan request, and give staff a compact queue for approvals and check-ins. It also serves as a practical example of a React client communicating with a Node and Express API, with Mongoose schemas prepared for MongoDB-backed storage.

Scope and Deliverables

The submitted application includes a dashboard, equipment catalogue, request queue, expected returns view, equipment request form, and Express API endpoints for health, equipment, request creation, and request status updates. The project includes a README, environment variable example, and this report, requirements specification, and interface preview PDF.

Authentication, server-enforced roles, audit history, and production database provisioning are outside the current prototype scope. Demonstration records are illustrative and should not be replaced with real student information.

Methodology

The work follows a small iterative build: define the laboratory checkout workflow, model equipment and loan request fields, create the React views, add API routes and validation, and verify the client against the local API. The interface uses sample records so its principal states can be reviewed without a configured database. MongoDB persistence activates when MONGODB_URI is set in the server environment.

Technology stack

- React 19 and Vite for the single-page client.
- Node.js and Express for the JSON API.
- MongoDB and Mongoose schemas for optional persistent storage.
- Lucide icons and responsive CSS for interface components.

Project Activities

- Defined the equipment fields: name, category, asset tag, status, and location.
- Created the overview dashboard with inventory counts, recent requests, and activity summary.
- Added equipment filters and search, a request form, and request status actions.

- Implemented API input checks for equipment and loan requests, status allow-list validation, health reporting, and safe error responses.
- Configured a demo-memory fallback and documented how to enable MongoDB persistence through an environment variable.
- Prepared project documentation and interface previews for review.

Results and Findings

The local build produces a responsive dashboard and separate catalogue, request, and return views. The API reports whether MongoDB is connected or whether it is running in demo-memory mode. The current sample flow demonstrates the interface and core request lifecycle, but it is not a production-ready multi-user service: the demo mode resets when restarted, and the prototype does not include authentication or authorization.

Conclusion

LabDesk demonstrates a focused equipment checkout flow using a MERN-style architecture. It makes the catalogue, request queue, and return step visible in one interface. The next work should add sign-in and role checks, test each API path against a managed MongoDB deployment, protect secrets, and verify the complete workflow on the deployed URL before the project is used with real people or data.

Appendix: Reference and use limits

Reference project: LabFlow by Milan K B, https://github.com/milan-kb/LabFlow (MIT license). LabDesk is an independent implementation informed by the problem area and general workflow. This report does not claim an internship placement, user study, or production deployment. Interface preview pages contain illustrative sample records.